

AI-Driven Workflows for QA Practice (Draft Proposal)



Page is about?

Objective

To identify repeatable, automation-prone workflows across QA that can be optimized or co-owned by AI agents (SASVA, Copilot, or ChatGPT), thus enabling faster delivery, leaner regression, and improved quality engineering productivity.

Summary: Where AI / Agentic AI Can Help QA Teams

Area	Workflow	AI Agent Use Case	Tool	Status
Test Case Creation	BDD-style scenario drafting	Auto-generate Given-When-Then test cases from JIRA stories	SASVA, ChatGPT	Ready
Automation Support	Convert manual test cases to code	Generate Java + Cucumber snippets with reusables	SASVA, Copilot	Pilot
Regression Maintenance	Detect flaky tests	Use execution trends to flag flaky or obsolete tests	SASVA	POC
Defect RCA	Auto-summarize root causes	Suggest RCA hints using past logs and test data	SASVA	Pilot
JIRA Story Creation	Workflow automation	Use prompts to create or triage JIRA items based on failure analysis	ChatGPT + JIRA API	Feasible
Report Summarization	Sprint QA dashboards	Generate module-wise test summary and failure trends	SASVA, ChatGPT	POC
Test Execution Analysis	Jenkins failure grouping	Group and explain regression failures by tag/module	SASVA	Ready
Knowledge Bot	QA onboarding help	"Where is the DB query for X?" Instant answer with link	SASVA	Live (Internal)
Test Coverage Validation	GitLab MR checks	Compare code changes vs test coverage	SASVA + GitLab API	Future
Cross-Tool Sync	Multi-tool integration	Sync data across qTest, Jenkins, GitLab, BrowserStack	SASVA API connectors	Planned

End-to-End AI Workflows (Agentic)

Here are proposed **end-to-end workflows** where a Virtual QA Assistant (like SASVA or a GenAI agent) can act independently or semi-independently:

1. JIRA Story to Test Coverage Flow

- AI reads JIRA story (EPIC or task).
- Suggests 5–10 BDD test scenarios.
- Drafts automation skeleton in Java/Cucumber.
- Adds tags, priorities, and edge cases.
- Pushes draft to qTest.

Outcome: 3–4 hours of manual effort reduced to <30 mins.

2. Regression Failure RCA Summary JIRA Defect

- Jenkins reports regression failure.
- AI parses logs detects pattern suggests root cause.
- Auto-generates RCA with probable impacted module.
- Drafts JIRA bug with linked history + fix hints.

Outcome: 30–60 mins saved per issue.

3. GitLab MR Test Coverage Check

- MR is created by Dev.
- AI reads changed files maps to test cases via tags.
- Suggests missing coverage or redundant tests.
- Adds comment in MR for test owner to take action.

Outcome: Faster code-review, better test coverage assurance.

4. Sprint Summary Generator

1. Before sprint review, AI reads last 2 weeks' Jenkins + qTest data.
2. Groups passed, failed, flaky, skipped cases by tag/module.
3. Generates confluence-ready table with insights.

Outcome: 100% automation of manual QA summary report.

Integration Targets

System	Actionable via API?	AI Agent Notes
qTest	Test case CRUD, lookup	Already mapped in SASVA
JIRA	Create/update stories	Easy integration with AI prompts
Jenkins	Logs, test results	Log summarizer + trend builder
GitLab	MR metadata, commit diff	Coverage bot (planned)
BrowserStack	Screenshots, logs	Future AI-driven screenshot analysis